



- AMAZONS -
Projet de programmation impérative

Saad KIOUEH
Mohamed Taha SANDI
Adib JOUAHRI
Mohamed Dyaé CHELLAF

Encadré par M. POPOV
Département informatique S6
Mars 2023 - Mai 2023

Table des matières

1	Introduction	2
1.1	Présentation du sujet	2
1.2	Objectif du projet	2
2	Cadre de travail	3
3	Organisation du projet	4
4	Problématique	5
5	Solutions algorithmiques et implémentation du code	7
5.1	Implémentation des graphes - Matrice d'adjascence	7
5.2	Interface Client - Serveur	8
5.2.1	Client	8
5.2.2	Serveur	8
5.3	Une partie de jeu des AMAZONS	9
5.4	Stratégies des joueurs	10
6	Etude algorithmique et complexité	12
6.1	Affichage des graphes	12
6.1.1	Le <i>Display</i> du graphe carré (<i>display_normal_board</i>)	12
6.1.2	Le <i>Display</i> du graphe donut (<i>display_donut_graph</i>)	12
6.2	Jeu aléatoire	13
7	Tests pour le code	13
8	Conclusion	14

1 Introduction

1.1 Présentation du sujet

Le jeu **AMAZONS** (apparu en Argentine en 1988) est un jeu de dames qui se joue sur un plateau d'échecs, incluant deux joueurs dont chacun d'eux dispose d'un nombre donné d'**amazones**, et cherche à maximiser ses possibilités de mouvement (aspect offensif) tout en minimisant ceux de l'adversaire (aspect défensif).

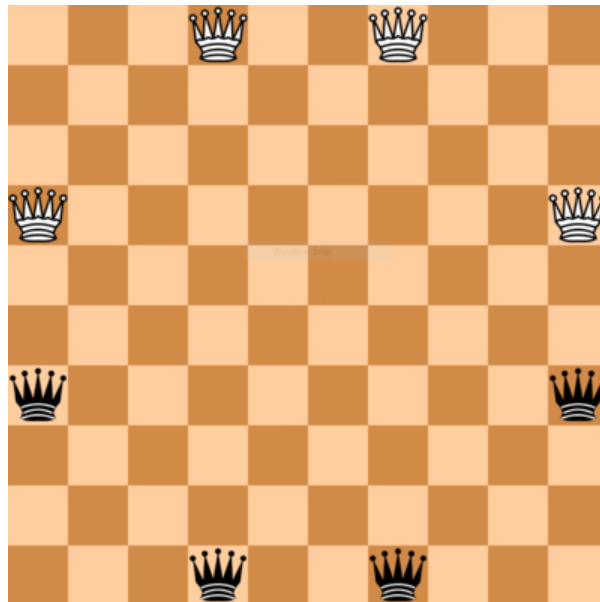


Figure 1: Plateau de jeu des **amazones**

Les deux joueurs jouent à tour de rôle, chacun d'eux, à son tour, déplace une de ses pièces (dites **QUEENS**) et tire une flèche dans une case qui lui est accessible. Une fois la flèche tirée, la case butée devient inaccessible pour les deux joueurs. La partie s'achève une fois l'un des deux joueurs ne peut plus déplacer aucune de ses pièces, il est donc déclaré perdant, et son adversaire déclaré vainqueur.

1.2 Objectif du projet

L'objectif de ce projet est la réalisation du déroulement de ce type de jeu et la mise en place des stratégies pour jouer aussi efficacement que

possible, et tout cela en adoptant l'esprit de la programmation impérative (en utilisant comme langage : **C**) comme le style unique et principale de la mise en place des divers algorithmes.

2 Cadre de travail

Dans cette partie, nous allons présenter les différents outils et méthodes utilisés afin de réaliser notre projet, dont la version finale doit être déposée dans le répertoire personnel de l'équipe sur le site de la Forge de notre école. Tout d'abord, nous avons eu à notre disposition 7 séances de travail, d'une durée de 4h20 chacune, sous la supervision de M.POPOV.

Le travail était à travers le git. Cet outil permet une gestion plus facile pour des projets. Il propose différentes fonctionnalités, comme l'historique des différents états du projet, une possibilité d'effectuer des dépôts avec actualisation des différents fichiers. Une fois le dépôt effectué, l'autre membre pourrait récupérer le travail réalisé au cours de la séance.

Pour le développement du projet, les fichiers sont organisés de la manière suivante:

- Un fichier **Makefile** où figurent un ensemble de commandes qui servent à compiler les fichiers sources et à générer les bibliothèques dynamiques.
- Un répertoire **src** qui contient les fichiers source.
- Un répertoire **tst** qui regroupe les tests pour l'ensemble des fonctions implémentées.
- Un répertoire **Install** qui contient les librairies dynamiques du projet.
- Un fichier **README.md** où figure des informations sur les commandes de compilation et d'exécution, ainsi que des informations sur la propriété des codes et les noms (des auteurs et de l'encadrant).

3 Organisation du projet

Afin de présenter l'architecture de notre projet ainsi que l'organisation des fichiers et leur regroupement, voici un diagramme des dépendances de notre projet:

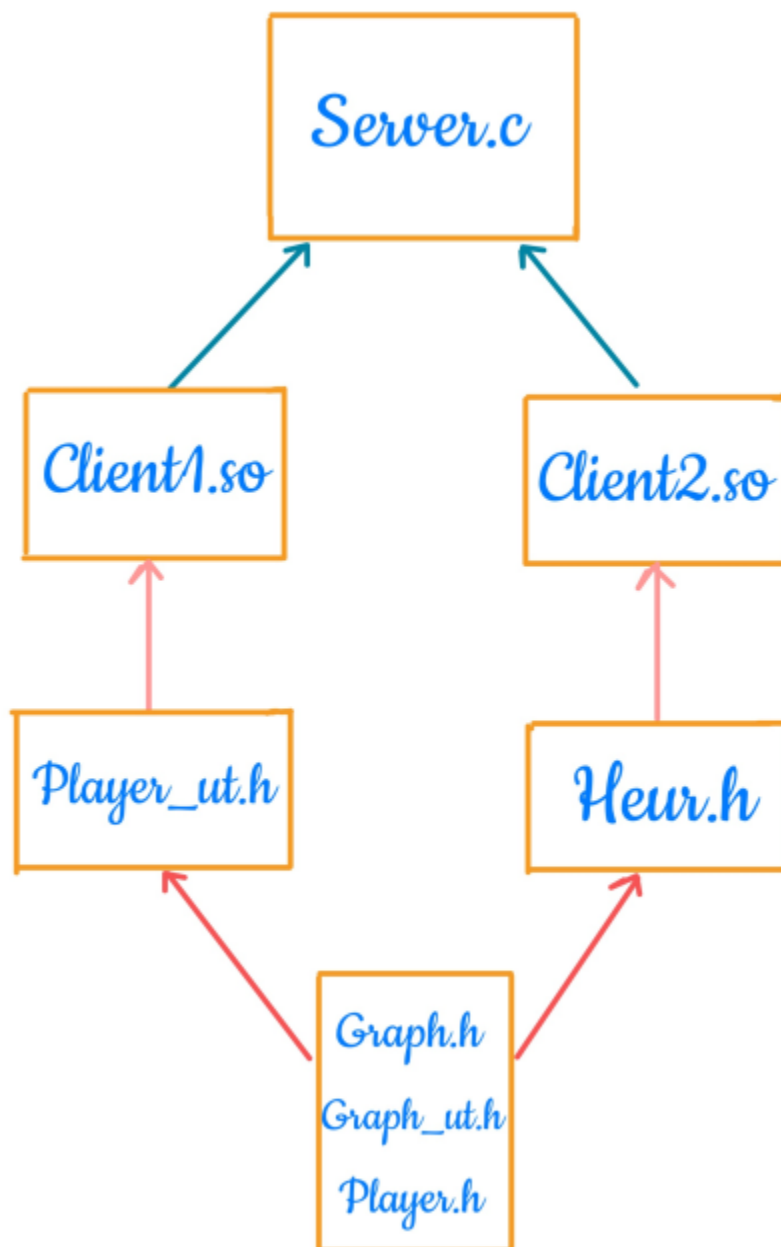


Figure 2: Graphe de dépendances

4 Problématique

La problématique principale de ce projet se constitue elle-même en deux sous-problèmes:

- Tout d'abord, la création d'un **serveur** de jeu qui organise une partie de jeu, faisant intervenir chaque joueur (client) à son tour en lui envoyant les coups joués par son adversaire, enregistrant le coup du joueur à son tour et notifiant les clients de la fin de la partie, tout en garantissant un déroulement de partie équitable entre les joueurs et sans possibilité de triche en testant la validité de chaque coup reçu de la part d'un joueur.
- Ensuite, la création d'un **client**. Le but ici était de créer des clients capables de jouer et de communiquer à travers tout serveur, et donc une cohérence entre les clients.

Il s'est posé aussi en parallèle le problème de créer des plateaux de jeu différents sous formes de graphes différents de tailles variables tout en respectant le fichier header ***graph.h*** et ***move.h*** qui imposaient des structures *graph_t* et *move_t*.

Le sujet du projet nous propose de créer 4 types de graphes dont 2 qui ont été implémenté (Carré et Donut), chacun est caractérisé par une longueur (notée m) et un nombre de sommets (*num_vertices*):

- Graphe carré
- Graphe donut
- Graphe trèfle
- Graphe en huit

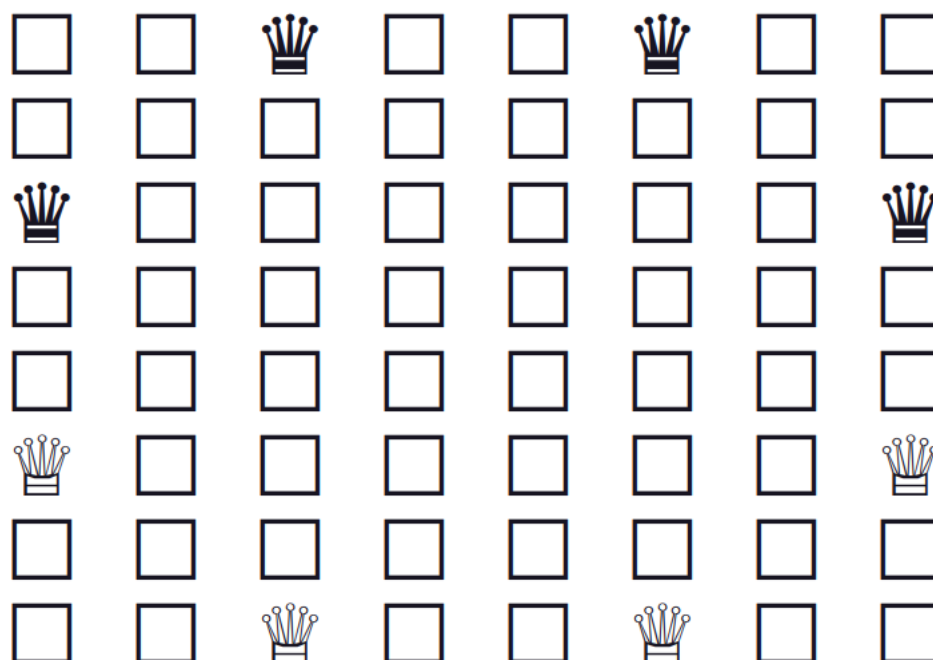


Figure 3: Graphe carré à l'état initial

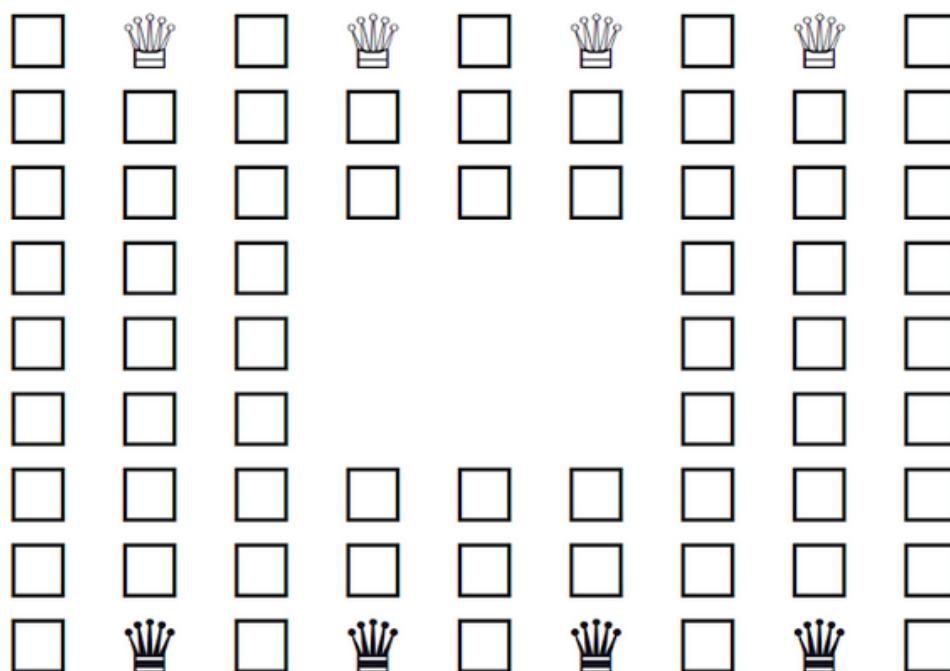


Figure 4: Graphe donut à l'état initial

5 Solutions algorithmiques et implémentation du code

5.1 Implémentation des graphes - Matrice d'adjascence

Une partie de jeu se déroule dans l'une des grilles mentionnées précédemment. Ces grilles se différencient, on peut avoir des obstacles (cases vides) qui ne contiennent pas de sommets comme on peut trouver des cases liées avec d'autres cases. Chaque sommet a des arrêts qu'avec les sommets qui l'entourent, et au niveau de la matrice d'adjascence, chaque lien entre deux sommets est représenté par l'énumération de direction partant du sommet source vers le sommet destination.

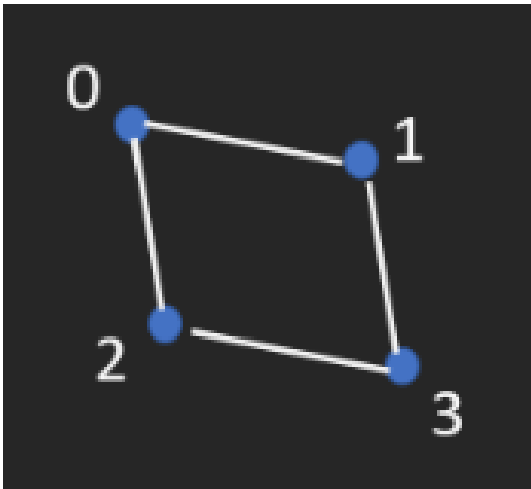


Figure 5: Graphe à 4 sommets

```
num_vertices = 4
t = [[ 0 7 5 0 ]
      [ 3 0 0 5 ]
      [ 1 0 0 7 ]
      [ 0 1 3 0 ]]
```

Figure 6: Matrice d'adjascence (tirée de la page du sujet du projet)

Dans ce cadre, on introduit les fichiers qui manipulent les graphes, dans le répertoire **src**, et qui peuvent être classés en deux parties principales:

- **Fichiers pour le graphe carré:** *Normal_graph.h* & *Normal_graph.c*
- **Fichiers pour le graphe donut:** *Donut_graph.h* & *Donut_graph.c*

Sans oublier un fichier *display.c* (avec son header) où est implémentée la fonction servant principalement à afficher les graphes.

5.2 Interface Client - Serveur

5.2.1 Client

Pour assurer la cohérence entre les joueurs, un header ***player.h*** nous est initialement fourni. Ainsi chaque joueur doit contenir les fonctions prototypes suivantes définies dans ce fichier:

- ***get_player_name***: retourne le nom du joueur.
- ***initialize***: initialise un joueur.
- ***play***: retourne le coup joué par le joueur en tenant en compte le coup qui le précède.
- ***finalize***: libère la mémoire précédemment allouée au joueur.

5.2.2 Serveur

En guise de rappel, la tâche principale du serveur est de garantir une équité entre les joueurs (clients), d'assurer un bon déroulement de la partie de jeu ainsi que de décider de la fin de cette dernière. Pour cela, on a décidé de lui associer un fichier ***server.c*** qui manipule principalement le serveur à travers les fonctions suivantes:

- ***next_player*** qui retourne l'ID du joueur suivant.
- ***isGameOver***, un booléen qui décide si la partie de jeu est terminée ou non.
- ***update_server*** qui met les tableaux des *queens* et des *arrows* sous la dernière version.

5.3 Une partie de jeu des AMAZONS

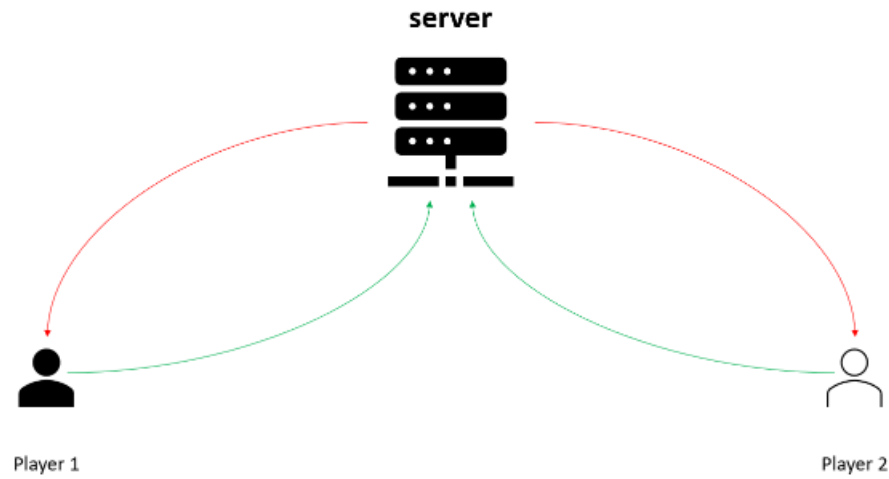


Figure 7: Plateau de jeu des **amazones**

Au départ, les différents paramètres de la partie tels que le type et la taille du graphe de jeu sont chargés à l'aide de la fonction ***"get_opt"***. Ensuite, le serveur initialise les joueurs grâce à la fonction ***initialize*** en leur attribuant un ID et une copie de jeu. Ainsi, la partie de jeu peut commencer.

Lorsque c'est au tour du premier joueur de jouer, il choisit son coup. Étant donné que le jeu se déroule indépendamment entre le joueur et le serveur, le joueur met à jour ses propres données (tableaux de reines et de flèches), puis transmet sa proposition au serveur. Celui-ci vérifie alors la validité du coup et met à jour ses informations locales. Il envoie ensuite le coup joué par le premier joueur au deuxième joueur, et ainsi de suite.

La partie prend fin si l'un des joueurs ne peut plus faire bouger ses pièces ou s'il transmet un coup invalide au serveur. Il est donc déclaré perdant.

5.4 Stratégies des joueurs

- Stratégie de jeu aléatoire

```
Function random_play
begin
  move <- initialize_empty_move()
  move.queen_src <- choose_free_random_queen()
  move.queen_dst <- choose_random_index(g.num_vertices)
  move.arrow_dst <- choose_random_index(g.num_vertices)

  while (!move_validity(move))
    move.queen_dst <- choose_random_index(g.num_vertices)
    move.arrow_dst <- choose_random_index(g.num_vertices)

  return move
end
```

Cet algorithme est conçu pour générer des mouvements aléatoires dans un jeu. Tout d'abord, il identifie toutes les pièces ayant au moins une possibilité de se déplacer. Ensuite, il en choisit une au hasard. Pour cette pièce sélectionnée, il génère deux indices aléatoires correspondant à la destination de la pièce et à la position de la flèche.

Une fois les indices générés, l'algorithme vérifie si le mouvement construit à partir de ces indices est valide. Si c'est le cas, le mouvement est retourné et le programme s'arrête. Si ce mouvement n'est pas valide, l'algorithme continue en tentant de trouver un nouveau coup valide à partir de nouveaux indices aléatoires.

Ce processus se répète jusqu'à ce qu'un mouvement valide soit trouvé. Cela permet de générer des mouvements aléatoires dans le jeu de manière efficace et sans risque de boucles infinies dans les coups invalides.

- Stratégie de mobilité

```
Function play
  amplitude <- amplitude_between_players(player1, player2);
  if (amplitude <= 1)
    return defensive_move()
  else
    return attack_move()
```

Cette fonction est un algorithme de prise de décision pour un joueur dans la partie de jeu.

Tout d'abord, il calcule l'amplitude entre les joueurs "player1" et "player2". L'amplitude est une mesure de distance entre les deux joueurs.

Si l'amplitude est inférieure ou égale à 1, cela signifie que les deux joueurs sont assez proches en termes de distance, et donc le joueur devrait adopter une stratégie défensive.

En revanche, si l'amplitude est supérieure à 1, cela signifie qu'il y a une différence significative de distance entre les deux joueurs, et donc le joueur devrait adopter une stratégie offensive pour tirer profit de cette différence.

En résumé, cette fonction "play" est un algorithme simple de prise de décision pour un joueur qui choisit sa stratégie de jeu en fonction de l'amplitude entre les joueurs.

6 Etude algorithmique et complexité

Au fur et à mesure de notre avancement lors de l'implémentation du code, on a été mené à créer des fonctions différentes, dont certaines ne sont pas aussi intéressantes pour l'étude de complexité que d'autres. Pour ce faire, on a fait un tri sur l'ensemble des fonctions et on a filtré les plus intéressantes pour cette étude:

- Affichage des graphes
- Jeu aléatoire (Stratégie I)
- Mobilité (Stratégie II)

6.1 Affichage des graphes

6.1.1 Le *Display* du graphe carré (*display_normal_board*)

Prenons comme notation, le nombre entier n représentant la taille du plateau de jeu (*board_size*).

La complexité de ce code est de $O(n)$. Le code comporte une boucle *for* qui itère sur chaque case du plateau de jeu et effectue des opérations en temps constant. La fonction *is_in_array* est appelée à chaque itération pour vérifier si la case actuelle contient une reine ou une flèche, mais elle a une complexité en temps constante car elle utilise un tableau de hachage pour stocker les éléments. Par conséquent, la complexité totale de la fonction *display_normal_board* est linéaire en fonction de la taille du plateau de jeu.

6.1.2 Le *Display* du graphe donut (*display_donut_graph*)

Prenons comme notation, le nombre entier m représentant le nombre de sommets du graphe.

La complexité de ce code est de $O(m)$. La boucle *for* parcourt chaque sommet du graphe une seule fois et effectue une opération constante pour chaque sommet. Le calcul de la taille de la table et du quotient est

également une opération constante. Par conséquent, la complexité totale est linéaire par rapport au nombre de sommets du graphe.

6.2 Jeu aléatoire

Tel qu'il est écrit, cet algorithme a une complexité variable, dépendant de l'avancement de la partie, ce qui a rendu l'étude de sa complexité plus intéressante. Donc pour éviter les risques d'une boucle infinie, on a décidé de fixer le nombre de fois où on accède à la boucle *while*. Une fois ce nombre excédé, on retourne un mouvement simple qui consiste à trouver la case vide la plus proche, cette dernière devient notre "*queen_dest*" et notre "*arrow_dest*" n'est autre que notre position initiale.

7 Tests pour le code

Les tests que nous avons implémentés sont des tests simples mais indispensables pour s'assurer du bon fonctionnement des fonctions les plus complexes, notamment pour l'implémentation des joueurs et du serveur. Nous avons commencé d'abord par tester l'implémentation du graphe *donut* en créant un graphe de taille 9 à l'aide de la fonction *donut_graph*. Ensuite, nous avons vérifié que le graphe obtenu contient les bonnes relations entre les différentes cases du plateau. Par ailleurs, nous avons testé les fonctions d'accès aux matrices *gsl* au format compressé, ainsi que les fonctions auxiliaires permettant de calculer le coup valide dans le fichier *player2.c*

8 Conclusion

Le projet **AMAZONS** de programmation impérative nous a présenté différents avantages et nous a offert l’opportunité de s’améliorer en programmation sur plusieurs niveaux.

D’une part, et vu que c’est un projet d’algorithmique et de programmation, nous avons appris à faire le lien entre la programmation en langage **C** et l’algorithmique des graphes à travers l’implémentation des plateaux de jeu qui sont sous forme de graphes. Aussi, cette implémentation nous a permis de découvrir la bibliothèque **GSL**.

D’autre part, ce projet a renforcé nos connaissances et nos compétences au niveau de la gestion dynamique de la mémoire, ainsi que de se familiariser avec l’utilisation des nouveaux outils d’édition de lien appris lors du module PG106.

Aussi, nous ne pourrions pas clôturer le rapport de ce projet sans faire référence à celui du semestre précédent. En effet, le projet du semestre 5 ("**Manṣūba**") était pour nous, notre toute première expérience de programmation sur un projet et qui nous a permis d’apprendre des nouvelles techniques de programmation et de nouvelles astuces de manipulation du dépôt *git*. Ces techniques et ces astuces ont été mises en œuvre et bien exploités par tous les membres de l’équipe, et la structure du projet **Manṣūba** nous a fourni une inspiration remarquable pour l’implémentation du code du projet **AMAZONS**.

Enfin, le travail en équipe au sein de ce projet s’est démarqué par une forte dynamique et une volonté de cohérence permanente.